

Bir gereksinimin anlamını görünür kılmak

Kurulum, FRETish, anlam diyagramı ve iki farklı yorumun karşılaştırılması

Bu rehber, tek bir gereksinimin belirsiz doğal dilden FRET'in işleyebildiği bir ifadeye nasıl dönüştürüldüğünü gösterir. Örnek, indirilen NASA FRET kaynak kodunun kendi ayrıştırıcısı ve anlambilim üreticisiyle işlenmiştir.

```
when request the ResponseSystem  
shall within 3 ticks satisfy response
```

Öğreneceğiniz sonuç: Talep ne zaman yükümlülük başlatır, yanıt ne zamana kadar gelmelidir ve bir kelime değişince davranış nasıl değişir?

Bu pakette	Durum
FRET masaüstü arayüzü ve yerel veri tabanı	Windows üzerinde yerel derleme; arayüz doğrulaması aşağıdaki ekranlarda.
FRETish ayrıştırma ve formül üretimi	REQ-001 ve REQ-002 gerçek FRET derleyicisinden geçirildi.
Örnek proje	NASA-FRET/tutorial/FRET_Tutorial_TR.json
LTLSIM ve realizability	Harici analiz araçları kurulmadı. Bu rehberdeki iz çizimleri açıklayıcıdır; LTLSIM çalıştırma sonucu değildir.

Hızlı başlangıç

- Çalışma klasöründeki **NASA-FRET/FRET-Baslat.cmd** dosyasına çift tıklayın.
- Requirements** listesinde **REQ-001** kimliğine tıklayın.
- Açılan özette kalem simgesine basın; düzenleyicide **SEMANTICS** düğmesini seçin.

Hazırlanma: 25 Eylül 2026. Kaynak: NASA-SW-VnV/fret, master arşivi, paket sürümü 3.0. Bu belge NASA'nın resmî eğitimi değildir; indirilen sürüm üzerinde hazırlanmış Türkçe uygulama rehberidir.

Önce anlamı belirleyin

Başlangıç cümlesi: **“Kullanıcı talep gönderdiğinde sistem kısa sürede yanıt vermelidir.”** FRET’in görevi, “kısa” sözcüğüne kendi başına bir süre atamak değildir. Aşağıdaki kararları analist verir ve kayıt altına alır.

Açık konu	Bu eğitimde verilen karar
Talep nedir?	request: Boolean giriş. FALSE değerinden TRUE değerine geçiş, yeni talep olayıdır. Başlangıçta TRUE olması da tetikler.
Yanıt nedir?	response: Boolean çıkış. Talebin alındı bildirimi değil, işlemin sonucunun hazır olmasıdır.
Kısa süre ne demek?	En fazla 3 zaman adımı; tetikleme anı da kabul edilir. Bu sayı eğitim varsayımdır.
Hangi kapsam?	Tüm yürütme. Belirli bir çalışma modu kısıtı yoktur.
Birden fazla talep?	Bu basit örnek talep kimliklerini eşlemez. Eşzamanlı çoklu talepler için ek modelleme gerekir.

FRETish alanlarına karşılığı

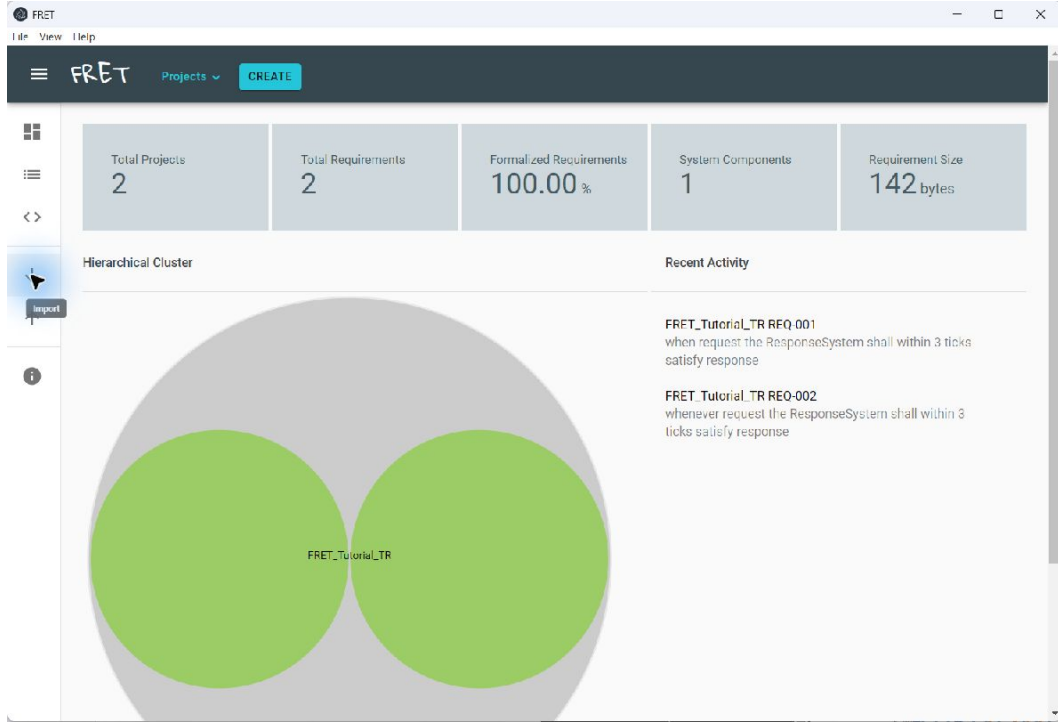
Alan	Örnekteki ifade	Anlam
Scope	Yazılmadı	Tüm yürütme
Condition	when request	Başlangıçta TRUE / yükselen kenar
Component	the ResponseSystem	Sorumlu sistem bileşeni
Modal	shall	Zorunluluk
Timing	within 3 ticks	0, 1, 2 veya 3 adım uzaklıkta
Response	satisfy response	response doğru olmalı

Zaman birimi: Bu sürümde FRET, seconds / milliseconds / ticks birimlerini fiziksel zamana dönüştürmez. 1 tick = 100 ms isteniyorsa, bu eşleme model ve doğrulama ortamında ayrıca kurulmalıdır. [3]

Örneği açın ve düzenleyin

Hazır örneği içe aktarmak için sol araç çubuğundaki **aşağı ok / Import** simgesini kullanın.

NASA-FRET/tutorial/FRET_Tutorial_TR.json dosyasını seçin; Windows dosya penceresinde **Aç** düğmesine basın. Proje otomatik yüklenir. Proje zaten yüklüyse tekrar içe aktarmanız gerekmez.



Şekil 1. Bu bilgisayarda çalışan FRET ve eğitim projesi. Ekrandaki yeşil durum, gereksinimin biçimselleştirildiğini gösterir; uygulamanın doğru çalıştığının kanıtı değildir.

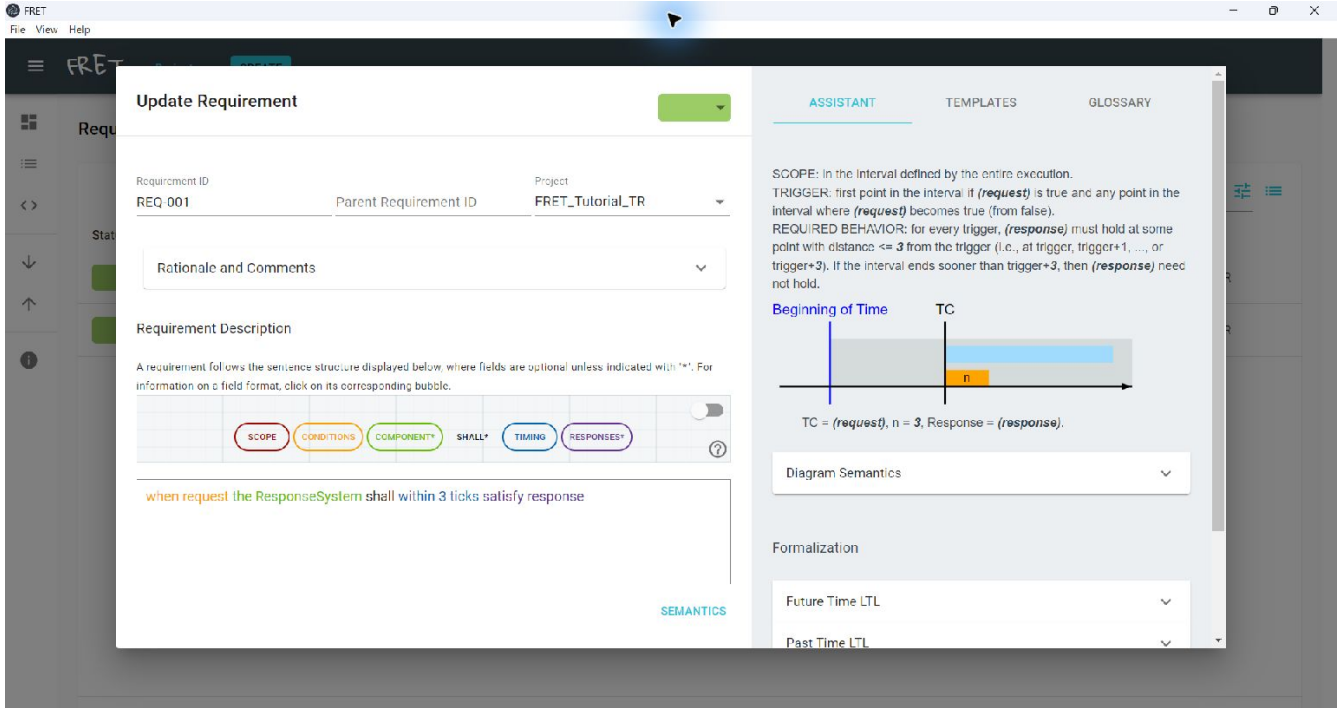
Aynı gereksinimi sıfırdan yazmak için

1. Projects menüsünden ayrı bir deneme projesi oluşturun.
2. **CREATE** ile yeni gereksinim açın.
3. Requirement ID alanına **REQ-001** yazın.
4. Aşağıdaki cümleyi metin alanına girin.
5. Rationale alanına “Yanıt = işlem sonucu; sınır = 3 tick” kararını kaydedin.
6. Ayırıştırma hatası olmadığını kontrol edin; **SEMANTICS** ile anlamı görün ve kaydedin.

when request the ResponseSystem shall within 3 ticks satisfy response

İfadeyi çift tırnak içine almayın. FRET'te tamamen tırnaklanmış metin, biçimselleştirilmeden saklanabilir. İngilizce anahtar sözcükler ve basit değişken adları kullanın; Türkçe açıklamayı gerekçe alanına yazabilirsiniz. [2]

SEMANTICS ekranını okuyun



Şekil 2. REQ-001 için gerçek FRET anlam görünümü. Alan renkleri, sözel açıklama ve zaman ilişkisi aynı gereksinime aittir.

Üç soruya yanıt arayın

SCOPE: Hangi yürütme aralığı geçerli? Burada tüm yürütme.

TRIGGER: Yükümlülük ne zaman başlar? request ilk noktada doğruysa veya sonradan FALSE → TRUE geçerse.

REQUIRED BEHAVIOR: Her tetiklemeden itibaren response, en geç üçüncü adımda en az bir kez doğru olmalıdır.

Diyagram: Gri alan kapsamı, açık mavi şerit koşulun doğru kaldığı bir aralığı, turuncu şerit yanıtın en az bir kez gerçekleşmesi gereken pencereyi gösterir. TC tetikleme anıdır; n = 3. Ayrıntılar için Diagram Semantics bölümünü açın.

FRET'in ürettiği sonsuz iz formülü

$$((G (((! request) \& (X request)) \rightarrow (X (F[0,3] response)))) \& (request \rightarrow (F[0,3] response)))$$

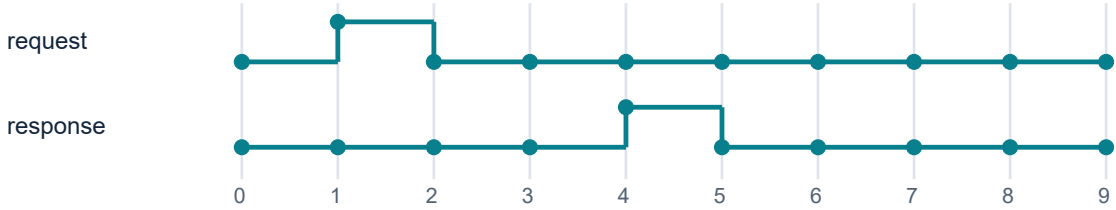
G: her zaman; **X:** sonraki adım; **F[0,3]:** şu andan başlayarak üç adım içinde en az bir kez. Formülün ilk bölümü yükselen kenarları, ikinci bölümü yürütmenin ilk noktasında request doğru olmasını kapsar.

Ekranın sözel açıklamasında sonlu aralığın erken bitmesine ilişkin ek cümle bulunur. Sonsuz iz formülünde LAST yoktur; sonlu iz formülünde vardır. Bu ayrımı 6. sayfadaki deneyle okuyun.

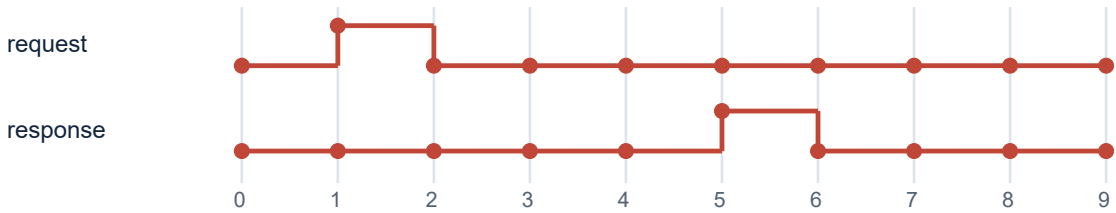
Resmi davranışa çevirin

Aşağıdaki çizimler FRET'in ürettiği anlamı açıklamak için hazırlanmıştır. Gerçek LTLSIM çıktısı değildir. request, t=1 anında tetikleniyor; bu nedenle izin verilen yanıt noktaları t=1, 2, 3 ve 4'tür.

A / Sınırdan yanıt: gereksinim sağlanır



B / Geç yanıt: gereksinim ihlal edilir



Deneme	Beklenen yorum
Yanıt t=1	Geçerli. "within" sıfır gecikmeyi de kabul eder.
Yanıt t=4	Geçerli. Tetiklemeden tam 3 adım sonra.
İlk yanıt t=5	İhlal. Tam gözlenen [1,4] penceresinde yanıt yok.
request hiç TRUE değil	Yükümlülük tetiklenmez. Bu, sistemin talep işlediğini kanıtlamaz.
response sürekli TRUE	Bu gereksinim sağlanabilir. "Yeni yanıt üretme" veya "yanıttan sonra sıfırlama" davranışı ayrıca istenmelidir.

Bir sonraki gereksinim sorusu

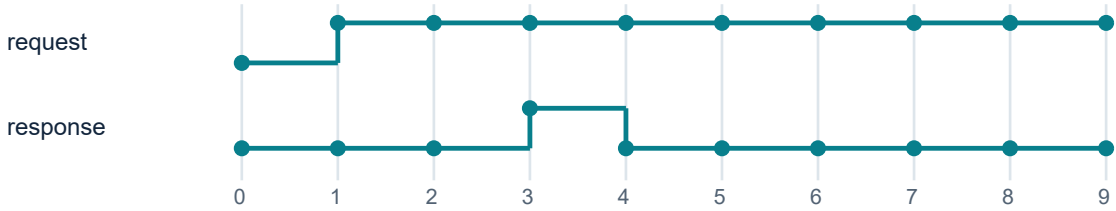
"Yanıtın yalnızca doğru olması yeterli mi, yoksa her talep için yeni bir yanıt olayı mı gerekli?" Bu farkı görsel ortaya çıkarır. İkinci anlam isteniyorsa talep/yanıt kimliği, sayaç veya protokol durumu eklemek gerekebilir.

Bir kelime değişince ne olur?

whenever request the ResponseSystem shall within 3 ticks satisfy response

REQ-002, REQ-001'in karşılaştırma alternatifidir. İkisini birlikte zorunlu gereksinimler olarak kabul etmek yerine ayrı ayrı yorumlayın. "whenever", request doğru kaldığı her noktada yeni bir yanıt penceresi başlatır.

Aynı iz / request yüksek kalıyor, tek yanıt t=3



İfade	Aynı izde sonuç
when / REQ-001	Yalnızca t=1 tetiklemesi vardır; t=3 yanıtı yeterlidir.
whenever / REQ-002	Örneğin t=4 yeni bir pencere açar. [4,7] içinde yanıt yoktur; ihlal oluşur.

REQ-002 için FRET'in ürettiği formül

(G (request -> (F[0,3] response)))

Kısa kayıt tuzağı

request t=1'de doğru olsun; response hiç doğru olmasın. Kayıt t=3'te biterse son tarih olan t=4 henüz görülmemiştir. Bu örneğin FRET sonlu iz anlambilimi, aralık erken bittiğinde yanıtın gerçekleşmesini zorunlu tutmaz. Dolayısıyla kısa izde olumlu değerlendirme, gerçek yanıtın alındığı anlamına gelmez.

Deney kararı: Her tetikleme için en az t+3 noktasını içeren kayıt alın. Kayıt erken bitmişse analiz raporunda "gözlem penceresi tamamlanmadı" durumunu ayrıca gösterin. Bu ifade önerilen raporlama kararıdır; FRET'in yeni bir sonuç kategorisi değildir.

Kendi gereksiniminize uygulayın

Adım	Yapılacak iş	Elde edilen kanıt
1. Ayır	Aktör, olay, tepki, kapsam ve süreyi ayır.	Metin → FRETish alan eşlemesi
2. Kararı kaydet	Belirsiz sözcükler için karar ver; gerekçesini yaz.	Varsayım ve paydaş kararı
3. Biçimselleştir	FRETish gir; ayrıştırmayı ve SEMANTICS açıklamasını kontrol et.	Formül ve anlam diyagramı
4. Ayırıcı iz kur	Bir yorumda geçen, diğerinde kalan davranış bul.	Somut yorum farkı
5. Sonraki soruyu seç	Davranışı değiştiren açık kararı sor.	Yeni bilgi / revizyon
6. Dışa aktar	Proje ve varsa değişken eşlemesini JSON olarak sakla.	Tekrar yüklenebilir proje

Wolfram tarzı görselleştirmeye bağlantı

FRET'te resim, tanımlanmış anlamın bir göstere midir. Projenizde de her hücre veya düğümü bir anlam alanına bağlayabilirsiniz. Örneğin “tetikleyici açık”, “süre kararlaştırıldı”, “test penceresi tamamlanmadı” farklı durumlardır. Hücrenin renk değiştirmesi, bir paydaş yanıtı veya kanıtla açıklanmalıdır.

İsteğe bağlı gelişmiş analiz

LTLSIM: NuSMV ve LTLSIM yerel simülatör bileşeni gerekir. Kurulduğunda SEMANTICS sonrasında SIMULATE açılır; Boolean izlerde noktaya tıklayarak değerler değiştirilir. Bu kurulumda bu arka uç kurulmadı. [4]

Realizability: Gereksinim kümesinin uygulanabilirliğini analiz eder; uygun çözümleyici ve giriş/çıkış değişken tanımları gerekir. Bir formülün üretilebilmesi, gereksinimlerin birlikte gerçekleştirilebilir olduğu anlamına gelmez. NASA'nın FSM örneği sonraki çalışma için uygundur. [5]

Şimdi sorulacak soru: Sizin sisteminizde açık kalan en kritik ayırım “ne zaman tetiklenir”, “hangi tepki istenir” veya “nasıl ölçülür” seçeneklerinden hangisi? İlk görseli bu ayrımı görünür kılacak şekilde seçin.

Kurulum kaydı ve kaynaklar

Bu bilgisayardaki kurulum

NASA deposunun master ZIP arşivi indirildi. Paket sürümü **3.0**. Node.js **20.19.0**, yalnızca NASA-FRET/runtime altında tutuldu. Ana npm bağımlılıkları kilit dosyasından kuruldu; masaüstü ve CLI bileşenleri derlendi. NASA'nın önerdiği Windows yolu WSL2'dir; burada WSL olmadığı için yerel Windows derlemesi kullanıldı.

Kurulum sırasında Unix temizleme komutlarını ve isteğe bağlı yerel simülatör derlemesini çalıştırmamak için npm yaşam döngüsü betikleri otomatik başlatılmadı. Electron kurulumu ve FRET derlemesi ayrı çalıştırıldı. Bu nedenle bu paket, tüm harici analiz araçları kurulmuş bir WSL ortamıyla eşdeğer değildir.

Dosya / klasör	Görevi
NASA-FRET/FRET-Baslat.cmd	Çift tıklayarak masaüstü uygulamasını açar.
NASA-FRET/fret-master	NASA kaynak kodu ve derlenmiş FRET.
NASA-FRET/runtime	Uygulamaya ayrılmış Node.js.
NASA-FRET/data	Gereksinim, model ve uygulama profil verileri.
NASA-FRET/tutorial/FRET_Tutorial_TR.json	İçe aktarılabilir, iki gereksinimli eğitim projesi.
NASA-FRET/tutorial/verified-semantics.json	Gerçek FRET derleyicisinden alınan metin, formül ve diyagram yolları.
tmp/fret-install	İndirme, bağımlılık ve derleme günlükleri.

Kaynaklar / erişim: 25 Eylül 2026

1. NASA FRET deposu ve kurulum
2. Gereksinim yazımı
3. Zaman alanı ve birim uyarısı
4. LTLSIM kullanımı
5. NASA sonlu durum makinesi örneği
6. Windows / WSL kurulum yolu
7. when / whenever ayrımı

Ekran görüntüleri: bu bilgisayardaki FRET çalıştırması. Davranış çizimleri ve Türkçe açıklamalar: bu rehber için üretildi. FRET kaynak lisansı: Apache 2.0. Kaynak arşivinin SHA-256 özeti kurulum kayıt dosyasında tutulur.